38

廉价磁盘冗余阵列（RAID）

使用磁盘时，我们有时希望它更快：I/O 操作很慢，可能成为整个系统的瓶颈。使用磁盘时，我们有时希望它更大：需要在线保存的数据越来越多，磁盘也越来越满。使用磁盘时，我们有时还希望它更可靠：磁盘一旦坏掉，如果数据没有备份，所有宝贵的数据就都没了。

关键问题：如何构建大容量、快速、可靠的磁盘
如何构建一个大容量、快速且可靠的存储系统？关键技术有哪些？不同方法之间要做哪些权衡？

本章介绍**廉价磁盘冗余阵列**（Redundant Array of Inexpensive Disks），也就是大家更熟悉的 **RAID** [P+88]。这项技术把多块磁盘组合起来协同工作，构建出更快、更大、更可靠的磁盘系统。这个术语是 20 世纪 80 年代末由加州大学伯克利分校的一组研究人员提出的（领头的是 David Patterson 教授和 Randy Katz 教授，还有当时还是学生的 Garth Gibson）；差不多就在那个时候，许多研究人员不约而同地想到了用多块磁盘构建更好的存储系统这个基本思路 [BG88, K86, K88, PB86, SG86]。

从外部看，RAID 就像一块磁盘：一组可以读写的块（block）。从内部看，RAID 却是个复杂的大家伙：它由多块磁盘、易失性（volatile）和非易失性（non-volatile）内存，以及一个或多个负责管理系统的处理器组成。硬件 RAID 非常像一个计算机系统，只不过专门用来管理一组磁盘。

与单块磁盘相比，RAID 有许多优点。一是性能：并行（in parallel）使用多块磁盘，可以大大缩短 I/O 时间。二是容量：大数据集需要大磁盘。最后，RAID 还能提高可靠性（reliability）：如果不用 RAID 技术，把数据分散到多块磁盘上，只要丢一块磁盘就会丢数据；而借助某种形式的**冗余**（redundancy），RAID 能承受一块磁盘的损失，照常运行，就像什么事都没发生一样。

令人惊讶的是，RAID 是**透明地**为使用它的系统提供这些优点的：在主机系统看来，RAID 就是一块大磁盘。透明的妙处当然在于，你可以直接把一块磁盘换成 RAID，一行软件都不用改；操作系统和客户端应用程序无须修改就能照常运行。这样，透明性大大提高了 RAID 的**可部署性**（deployability），用户和管理员可以直接用上 RAID，不必担心软件兼容性问题。

提示：透明有利于部署

给系统添加新功能时，应该始终想一想能否**透明地**（transparently）添加，也就是不要求系统其他部分做任何改动。如果要求完全重写现有软件（或者对硬件做根本性的改动），这个想法就很难产生影响。RAID 就是绝佳的例子，它的成功无疑有透明性的一份功劳：管理员可以装一个基于 SCSI 的 RAID 存储阵列来代替 SCSI 磁盘，系统的其他部分（主机、操作系统等）不用做一丁点改动就能用上它。由于解决了这个**部署**（deployment）问题，RAID 从第一天起就更容易取得成功。

下面我们讨论 RAID 的几个重要方面。先讲接口和故障模型（fault model），然后讨论如何从三个重要维度评估一种 RAID 设计：容量、可靠性和性能。最后，再讨论其他一些对 RAID 的设计和实现很重要的问题。

38.1 接口与 RAID 内部结构

在上层的文件系统（file system）看来，RAID 就像一块很大、（但愿）很快、（但愿）很可靠的磁盘。和单块磁盘一样，它表现为一个由块组成的线性数组，文件系统或其他客户端可以读写其中的每一块。

文件系统向 RAID 发出一个*逻辑 I/O* 请求后，RAID 必须在内部算出要访问哪块（或哪几块）磁盘才能完成请求，然后发出一个或多个*物理 I/O* 来完成它。这些物理 I/O 具体是什么样子，取决于 RAID 的级别，后面我们会详细讨论。举个简单的例子，假设某个 RAID 为每个块保存两个副本（分别放在不同的磁盘上）；向这样的**镜像**（mirrored）RAID 系统写数据时，RAID 每收到一个逻辑 I/O，就要执行两个物理 I/O。

RAID 系统通常做成一个独立的硬件盒子，通过标准接口（例如 SCSI 或 SATA）连到主机上。不过，RAID 内部相当复杂：它有一个运行固件（firmware）的微控制器，指挥 RAID 的运作；有 DRAM 之类的易失性内存，在读写数据块（data block）时用来缓冲；有的还带有非易失性内存，用来安全地缓冲写操作；甚至可能有专门的逻辑电路来做奇偶校验（parity）计算（这在某些 RAID 级别中很有用，后面也会看到）。从高层次看，RAID 就是一个高度专门化的计算机系统：它有处理器、内存和磁盘；只不过它运行的不是应用程序，而是专门用来操作 RAID 的软件。

38.2 故障模型

要理解 RAID、比较各种方法，心里必须先有一个故障模型。RAID 的设计目标是检测某几类磁盘故障并从中恢复；因此，确切知道会出现哪些故障，对于得出一个可行的设计至关重要。

我们假设的第一个故障模型相当简单，称为 **fail-stop** 故障模型 [S84]。在这个模型中，磁盘只可能处于两种状态之一：正常工作或者已经故障。正常工作的磁盘，所有块都可以读写。相反，磁盘一旦故障，我们就认为它永久丢失了。

 © 2008–23, ARPACI-DUSSEAU

fail-stop 模型的一个关键方面，在于它对故障检测的假设。具体来说，我们假设磁盘出了故障很容易检测出来。例如，在 RAID 阵列中，我们会假设 RAID 控制器的硬件（或软件）能立即发现某块磁盘出了故障。

因此，我们暂时不必担心更复杂的“静默”故障，比如磁盘数据损坏。也不必担心一块整体正常的磁盘上某个块变得无法访问的情况（有时称为 latent sector error）。这些更复杂（而且很遗憾，也更现实）的磁盘故障，我们后面再考虑。

38.3 如何评估 RAID

我们很快就会看到，构建 RAID 的方法有很多种。每种方法各有特点，值得逐一评估，以了解它们的优缺点。

具体来说，我们从三个维度评估每种 RAID 设计。第一个维度是**容量**（capacity）：给定 N 块磁盘，每块有 B 个块，RAID 的客户端能用到多少有效容量？如果没有冗余，答案是 $N \cdot B$；相反，如果系统为每个块保存两个副本（称为**镜像**，mirroring），有效容量就是 $(N \cdot B)/2$。其他方案（例如基于奇偶校验的方案）往往介于两者之间。

第二个评估维度是**可靠性**（reliability）：给定的设计能承受几块磁盘故障？按照我们的故障模型，只考虑整块磁盘故障的情况；在后面讲数据完整性（data integrity）的那一章里，我们会考虑如何处理更复杂的故障模式。

最后，第三个维度是**性能**（performance）。性能评估起来有些难度，因为它在很大程度上取决于磁盘阵列面对的工作负载（workload）。因此，在评估性能之前，我们先介绍几种应该考虑的典型工作负载。

接下来我们考虑三种重要的 RAID 设计：RAID 0 级，即条带化（striping）；RAID 1 级，即镜像；RAID 4/5 级，即基于奇偶校验的冗余。把这些设计各自称为一个“级别”（level），源自 Patterson、Gibson 和 Katz 在伯克利的开创性工作 [P+88]。

38.4 RAID 0 级：条带化

第一个 RAID 级别其实根本算不上 RAID 级别，因为它没有冗余。不过，RAID 0 级，也就是大家更熟悉的**条带化**（striping），可以作为性能和容量的一个绝佳上限，因此值得了解。

最简单的条带化（striping），就是像下面这样，把块依次**条带化**（stripe）到系统的各块磁盘上（这里假设阵列有 4 块磁盘）：

磁盘 0	磁盘 1	磁盘 2	磁盘 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

图 38.1：**RAID-0：简单条带化**

 © 2008–23, ARPACI-DUSSEAU

从图 38.1 可以看出基本思路：把阵列的块轮流（round-robin）分散到各块磁盘上。这种方法的设计目的，是在请求访问阵列中连续的大段数据时（例如大规模的顺序读），尽可能发挥阵列的并行性（parallelism）。我们把同一行的块称为一个**条带**（stripe）；因此，上面的块 0、1、2、3 位于同一个条带（stripe）中。

在这个例子中，我们做了一个简化假设：每块磁盘上只放 1 个块（比如每块 4 KB），然后就转到下一块磁盘。但并非一定要这样安排。例如，我们也可以像图 38.2 那样在各磁盘上排列块：

磁盘 0	磁盘 1	磁盘 2	磁盘 3	
0	2	4	6	条带深度：
1	3	5	7	2 个块
8	10	12	14	
9	11	13	15	

图 38.2：**条带深度更大的条带化**

在这个例子中，我们在每块磁盘上放两个 4 KB 的块，然后才转到下一块磁盘。因此，这个 RAID 阵列的**条带深度**（chunk size）是 8 KB，一个条带（stripe）就由 4 个大块（chunk）组成，共 32 KB 数据。

补充：RAID 映射问题

在研究 RAID 的容量、可靠性和性能特点之前，我们先插一段补充，讲讲所谓的**映射问题**（mapping problem）。所有 RAID 阵列都会遇到这个问题；简单地说，给定一个要读或写的逻辑块（logical block），RAID 怎么准确知道该访问哪块物理磁盘的哪个偏移位置？

对于这几种简单的 RAID 级别，要把逻辑块正确地映射到物理位置，用不着多复杂的方法。拿上面第一个条带化（striping）的例子来说，条带深度（chunk size）= 1 个块 = 4 KB。这时，给定逻辑块地址（logical block address）A，RAID 用两个简单的等式就能算出要访问的磁盘和偏移：

```
Disk   = A % number_of_disks
Offset = A / number_of_disks
```

注意，这些都是整数运算（例如 4 / 3 = 1，而不是 1.33333…）。我们用一个简单的例子看看这两个等式怎么用。假设在上面第一个 RAID 中，来了一个访问块 14 的请求。由于有 4 块磁盘，我们要找的磁盘就是（14 % 4 = 2）：磁盘 2。具体的块则是（14 / 4 = 3）：块 3。因此，块 14 应该在第三块磁盘（磁盘 2，从 0 开始编号）的第四个块（块 3，从 0 开始编号）上，它也确实就在那里。

你可以想一想，要支持不同的条带深度（chunk size），这两个等式该怎么改。试试看！不太难。

 © 2008–23, ARPACI-DUSSEAU

条带深度

条带深度（chunk size）主要影响阵列的性能。例如，条带深度（chunk size）小，意味着许多文件会条带化（striped）到许多磁盘上，从而提高读写单个文件时的并行性；但是，跨多块磁盘访问块的定位时间会增加，因为整个请求的定位时间，取决于各块磁盘上请求的定位时间中的最大值。

另一方面，条带深度（chunk size）大，会降低这种文件内的并行性，因此要靠多个并发（concurrent）请求才能获得高吞吐量（throughput）。不过，大的条带深度（chunk size）能减少定位时间；例如，如果一个文件能放进一个大块，因而只放在一块磁盘上，那么访问它的定位时间就只是一块磁盘的定位时间。

因此，确定“最佳”条带深度（chunk size）很难，因为这需要对磁盘系统面对的工作负载有大量了解 [CL95]。在接下来的讨论中，我们假设阵列的条带深度（chunk size）是一个块（4 KB）。大多数阵列使用更大的条带深度（chunk size，例如 64 KB），但对于下面讨论的问题，具体的条带深度（chunk size）无关紧要；所以为简单起见，我们就用一个块。

回到 RAID-0 分析

现在我们来评估条带化（striping）的容量、可靠性和性能。从容量的角度看，它是完美的：给定 N 块磁盘，每块 B 个块，条带化（striping）能提供 $N \cdot B$ 个块的有效容量。从可靠性的角度看，条带化（striping）也“完美”，不过是坏的那种完美：任何一块磁盘故障都会导致数据丢失。最后，性能非常出色：所有磁盘都能用上，而且往往是并行地处理用户的 I/O 请求。

评估 RAID 性能

分析 RAID 性能时，可以考虑两种不同的性能指标。第一种是*单请求延迟*（single-request latency）。了解单个 I/O 请求访问 RAID 的延迟很有用，因为它能揭示一次逻辑 I/O 操作内部能有多少并行性。第二种是 RAID 的*稳态吞吐量*（steady-state throughput），也就是许多并发请求的总带宽（bandwidth）。由于 RAID 常用在高性能环境中，稳态带宽至关重要，因此它将是我们分析的重点。

为了更详细地理解吞吐量，我们需要先确定几种要关注的工作负载。在这里的讨论中，我们假设有两种工作负载：**顺序**（sequential）负载和**随机**（random）负载。对于顺序负载，我们假设对阵列的请求是大块连续的；例如，一个（或一串）从块 x 开始、到块（x + 1 MB）结束、访问 1 MB 数据的请求，就视为顺序请求。顺序负载在许多环境中都很常见（想想在一个大文件里搜索某个关键词），因此很重要。

对于随机负载，我们假设每个请求都相当小，而且每个请求都访问磁盘上一个不同的随机位置。例如，一串随机请求可能先访问逻辑地址 10 处的 4 KB，然后是逻辑地址 550,000 处，再然后是 20,100 处，依此类推。一些重要的工作负载，比如数据库管理系统（DBMS）上的事务型（transactional）负载，就表现出这种访问模式，因此随机负载同样很重要。

当然，真实的工作负载没有这么简单，往往既有顺序的成分，也有看似随机的成分，还有介于两者之间的行为。为简单起见，我们只考虑这两种情况。

 © 2008–23, ARPACI-DUSSEAU

你也能想到，顺序负载和随机负载会让磁盘表现出截然不同的性能特点。顺序访问时，磁盘工作得最高效，花在寻道（seek）和等待旋转上的时间很少，大部分时间都在传输数据。随机访问（random access）时恰恰相反：大部分时间花在寻道和等待旋转上，花在传输数据上的时间相对很少。为了在分析中体现这种差别，我们假设磁盘在顺序负载下能以 S MB/s 的速率传输数据，在随机负载下则为 R MB/s。一般来说，S 远大于 R（即 $S \gg R$）。

为了确保我们理解了这种差别，来做个简单的练习。具体来说，根据下面给出的磁盘特性计算 S 和 R。假设顺序传输平均为 10 MB，随机传输平均为 10 KB。另外，假设磁盘特性如下：

平均寻道时间	7 ms
平均旋转延迟	3 ms
磁盘传输速率	50 MB/s

要计算 S，得先弄清楚一次典型的 10 MB 传输把时间花在了哪里。首先，寻道花 7 ms，然后旋转花 3 ms。最后开始传输；以 50 MB/s 的速率传输 10 MB，需要 1/5 秒，也就是 200 ms。因此，每个 10 MB 的请求要花 210 ms 才能完成。要计算 S，只需做个除法：

$$S = \frac{\text{数据量}}{\text{访问时间}} = \frac{10 \text{ MB}}{210 \text{ ms}} = 47.62 \text{ MB/s}$$

可以看到，由于传输数据花的时间很长，寻道和旋转的开销被摊薄了，S 非常接近磁盘的峰值带宽。

R 的计算方法类似。寻道和旋转的时间一样；接着计算传输时间（transfer time）：以 50 MB/s 的速率传输 10 KB，需要 0.195 ms。

$$R = \frac{\text{数据量}}{\text{访问时间}} = \frac{10 \text{ KB}}{10.195 \text{ ms}} = 0.981 \text{ MB/s}$$

可以看到，R 不到 1 MB/s，S/R 差不多是 50。

再回到 RAID-0 分析

现在来评估条带化（striping）的性能。前面说过，它总体上不错。例如，从延迟的角度看，单块请求的延迟应该和单块磁盘几乎一样；毕竟，RAID-0 只是把这个请求转给它的某一块磁盘而已。

从稳态顺序吞吐量的角度看，我们预计能获得系统的全部带宽。因此，吞吐量等于磁盘数量 N 乘以单块磁盘的顺序带宽 S。对于大量随机 I/O，我们同样能用上所有磁盘，从而获得 $N \cdot R$ MB/s。下面我们会看到，这两个值最容易计算，也可以作为和其他 RAID 级别比较时的上限。

38.5 RAID 1 级：镜像

条带化（striping）之后的第一个 RAID 级别是 RAID 1 级，也就是镜像。在镜像系统（mirrored system）中，我们只是给系统中的每个块多做几个副本；当然，每个副本应该放在不同的磁盘上。这样，我们就能承受磁盘故障。

在典型的镜像系统（mirrored system）中，我们假设 RAID 为每个逻辑块保存两个物理副本。下面是一个例子：

磁盘 0	磁盘 1	磁盘 2	磁盘 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

图 38.3：**简单的 RAID-1：镜像**

在这个例子中，磁盘 0 和磁盘 1 的内容完全相同，磁盘 2 和磁盘 3 也一样；数据在这两组镜像对（mirrored pair）之间做了条带化（striped）。实际上，你可能已经注意到，在各磁盘上放置块副本的方式有好几种。上面这种布局很常见，有时称为 **RAID-10**（或 **RAID 1+0**，**镜像的条带**，stripe of mirrors），因为它先组成镜像对（RAID-1），再在其上做条带化（RAID-0）；另一种常见的布局是 **RAID-01**（或 **RAID 0+1**，**条带的镜像**，mirror of stripes），它包含两个大的条带化（RAID-0）阵列，再在其上做镜像（RAID-1）。眼下我们只讨论上面这种布局的镜像。

从镜像阵列（mirrored array）读一个块时，RAID 有选择余地：两个副本读哪个都行。例如，如果向 RAID 发出读逻辑块 5 的请求，它既可以从磁盘 2 读，也可以从磁盘 3 读。不过，写一个块时就没有这种选择了：为了保证可靠性，RAID 必须把两个副本都更新。但要注意，这两个写操作可以并行进行；例如，写逻辑块 5 时，可以同时写磁盘 2 和磁盘 3。

RAID-1 分析

我们来评估一下 RAID-1。从容量的角度看，RAID-1 很昂贵：镜像级别（mirroring level）为 2 时，只能得到峰值有效容量的一半。N 块磁盘、每块 B 个块时，RAID-1 的有效容量是 $(N \cdot B)/2$。

从可靠性的角度看，RAID-1 表现不错。它能承受任意一块磁盘的故障。你可能还注意到，运气好的话，RAID-1 其实还能做得更好。想象一下，在上图中，磁盘 0 和磁盘 2 都坏了。这种情况下，数据并没有丢失！更一般地说，镜像级别（mirroring level）为 2 的镜像系统（mirrored system）肯定能承受 1 块磁盘故障，而视坏的是哪几块磁盘，最多能承受 $N/2$ 块磁盘故障。在实践中，我们一般不喜欢把这种事交给运气；所以大多数人认为镜像适合应对单块磁盘故障。

最后，我们来分析性能。从单个读请求的延迟来看，它和单块磁盘上的延迟相同；RAID-1 做的只是把读请求转给其中一个副本。写请求则有点不同：它要等两次物理写都完成才算完成。这两次写是并行的，因此时间大致相当于一次写的时间；但是，由于逻辑写必须等两次物理写都完成，它的耗时取决于两个请求中寻道和旋转延迟（rotational delay）较长的那个，因此（平均而言）会比写单块磁盘稍长一些。

为了分析稳态吞吐量（steady-state throughput），我们先从顺序负载开始。顺序写磁盘时，每次逻辑写都必须变成两次物理写；例如，写逻辑块 0 时（上图），RAID 在内部会把它同时写到磁盘 0 和磁盘 1。因此可以得出结论：顺序写镜像阵列（mirrored array）时能获得的最大带宽是 $\left(\frac{N}{2} \cdot S\right)$，也就是峰值带宽的一半。

 © 2008–23, ARPACI-DUSSEAU

补充：RAID 的 consistent-update problem

在分析 RAID-1 之前，我们先讨论一个任何多磁盘 RAID 系统都会遇到的问题，称为 **consistent-update problem** [DAA05]。向 RAID 写数据时，如果一次逻辑操作要更新多块磁盘，就会出现这个问题。这里假设讨论的是一个镜像磁盘阵列（mirrored disk array）。

假设一个写请求发给了 RAID，RAID 判定必须把它写到两块磁盘上：磁盘 0 和磁盘 1。RAID 先向磁盘 0 发出写请求，但就在 RAID 向磁盘 1 发出请求之前，发生了断电（power loss）或系统崩溃（system crash）。在这种不幸的情况下，我们假设发给磁盘 0 的请求完成了（而发给磁盘 1 的请求显然没有完成，因为它根本没发出去）。

这次不合时宜的断电造成的结果是，这个块的两个副本现在**不一致**（inconsistent）了：磁盘 0 上的副本是新版本，磁盘 1 上的是旧版本。我们希望两块磁盘的状态能**原子地**（atomically）改变，也就是说，要么两者都变成新版本，要么都不变。

解决这个问题的一般方法，是使用某种**预写日志**（write-ahead log），在动手之前先记下 RAID 将要做什么（即用某份数据更新两块磁盘）。采用这种方法，就能保证即使发生崩溃，也会得到正确的结果：运行一个**恢复**（recovery）程序，把所有未完成的事务（transaction）重放（replay）到 RAID 上，就能确保（在 RAID-1 的情况下）没有哪两个镜像副本（mirrored copy）是不同步的。

最后说一点：由于每次写都往磁盘记日志的开销高得令人无法接受，大多数 RAID 硬件都带有少量非易失性 RAM（例如由电池供电），在里面记这种日志。这样，就能保证更新的一致性，又不必付出往磁盘记日志的高昂代价。

遗憾的是，顺序读的性能也完全一样。有人可能会认为顺序读可以做得更好，因为它只需要读一个副本，而不是两个都读。不过，我们用一个例子来说明为什么这帮不上多大忙。假设我们要读块 0、1、2、3、4、5、6、7，并把读块 0 的请求发给磁盘 0，读块 1 的发给磁盘 2，读块 2 的发给磁盘 1，读块 3 的发给磁盘 3。接着，再分别把读块 4、5、6、7 的请求发给磁盘 0、2、1、3。有人可能会天真地以为，既然所有磁盘都用上了，我们就获得了阵列的全部带宽。

然而，事情并非（一定）如此。来看单块磁盘（比如磁盘 0）收到的请求。它先收到读块 0 的请求，然后收到读块 4 的请求（跳过了块 2）。实际上，每块磁盘都是每隔一个块才收到一个请求。磁盘转过被跳过的那个块时，并没有给客户端提供有用的带宽。因此，每块磁盘只能提供其峰值带宽的一半。所以，顺序读也只能获得 $\left(\frac{N}{2} \cdot S\right)$ MB/s 的带宽。

随机读是镜像 RAID（mirrored RAID）的最好情况。这时，我们可以把读请求分散到所有磁盘上，从而获得全部可能的带宽。因此，对于随机读，RAID-1 能提供 $N \cdot R$ MB/s。

最后，随机写的性能和你预想的一样：$\frac{N}{2} \cdot R$ MB/s。每次逻辑写都要变成两次物理写，因此，虽然所有磁盘都在工作，客户端感受到的却只有一半可用带宽。尽管对逻辑块 x 的一次写会变成对两块不同物理磁盘的两次并行（parallel）写，但大量小请求的带宽只能达到条带化（striping）时的一半。我们很快就会看到，能拿到一半的可用带宽其实已经相当不错了！

 © 2008–23, ARPACI-DUSSEAU

38.6 RAID 4 级：用奇偶校验节省空间

现在我们介绍另一种给磁盘阵列增加冗余的方法，称为**奇偶校验**（parity）。基于奇偶校验的方法试图少用一些容量，从而克服镜像系统（mirrored system）在空间上付出的巨大代价。不过，这样做也有代价：性能。

磁盘 0	磁盘 1	磁盘 2	磁盘 3	磁盘 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

图 38.4：**带奇偶校验的 RAID-4**

图 38.4 是一个五磁盘 RAID-4 系统的例子。我们为每个数据条带（stripe）增加了一个**奇偶校验块**（parity block，下面简称校验块），用来存放该条带中各块的冗余信息。例如，校验块（parity block）P1 存放的冗余信息，就是根据块 4、5、6、7 算出来的。

要计算奇偶校验，我们需要一个数学函数，让我们能承受条带（stripe）中任意一个块的丢失。简单的**异或**（XOR）函数就能很好地做到这一点。对于给定的一组位，如果其中 1 的个数是偶数，所有这些位的 XOR 结果就是 0；如果 1 的个数是奇数，结果就是 1。例如：

C0	C1	C2	C3	P
0	0	1	1	XOR(0,0,1,1) = 0
0	1	0	0	XOR(0,1,0,0) = 1

在第一行（0,0,1,1）中，有两个 1（C2、C3），因此所有这些值的 XOR 结果是 0（P）；同样，第二行只有一个 1（C1），因此 XOR 结果一定是 1（P）。有个简单的办法可以记住这一点：任何一行中 1 的个数（包括校验位在内）都必须是偶数（而不是奇数）；这就是 RAID 为了保证奇偶校验正确而必须维持的**不变式**（invariant）。

从上面的例子，你也许还能猜到如何利用奇偶校验信息从故障中恢复。假设标为 C2 的那一列丢了。要弄清这一列原来必定是什么值，我们只需读入这一行的其他所有值（包括 XOR 算出的校验位），就能**重建**（reconstruct）出正确的答案。具体来说，假设第一行 C2 列的值丢了（它是 1）；读这一行的其他值（C0 的 0、C1 的 0、C3 的 1，以及校验列 P 的 0），我们得到 0、0、1、0。因为我们知道 XOR 会让每一行中 1 的个数保持为偶数，所以就知道缺失的数据一定是：1。基于 XOR 的奇偶校验方案就是这样重建数据的！还要注意我们计算重建值的方法：只需把数据位和校验位放在一起做 XOR，和当初计算校验位的方法一样。

现在你可能会问：我们一直在说对这些位做 XOR，可从前面我们知道，RAID 在每块磁盘上放的是 4 KB（或更大）的块；怎么对一堆块做 XOR 来计算校验呢？其实这也很简单。只需对数据块的每一位做按位 XOR，再把每次按位 XOR 的结果放进校验块（parity block）中对应的位上。例如，如果块的大小是 4 位（是的，这还是比 4 KB 的块小得多，但你明白意思就行），它们可能是下面这个样子：

磁盘 0	磁盘 1	磁盘 2	磁盘 3	磁盘 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

图 38.5：**RAID-4 中的全条带写**

块 0	块 1	块 2	块 3	校验块
00	10	11	10	11
10	01	00	01	10

从图中可以看出，校验是对每个块的每一位分别计算的，结果放在校验块（parity block）中。

RAID-4 分析

现在来分析 RAID-4。从容量的角度看，RAID-4 在它所保护的每一组磁盘中，用 1 块磁盘存放校验信息。因此，一个 RAID 组的有效容量是 $(N-1)\cdot B$。

可靠性也很容易理解：RAID-4 能承受 1 块磁盘故障，多一块都不行。如果丢失的磁盘超过一块，就根本没有办法重建丢失的数据。

最后是性能。这次我们先分析稳态吞吐量（steady-state throughput）。顺序读能利用除校验盘（parity disk）以外的所有磁盘，因此能提供 $(N-1)\cdot S$ MB/s 的峰值有效带宽——这是简单的情况。

要理解顺序写的性能，首先要弄清楚它是怎么完成的。把一大块数据写到磁盘时，RAID-4 可以做一种简单的优化，称为**全条带写**（full-stripe write）。例如，设想块 0、1、2、3 作为一个写请求的一部分发给了 RAID（图 38.5）。

在这种情况下，RAID 可以直接算出 P0 的新值（对块 0、1、2、3 做 XOR），然后把包括校验块（parity block）在内的所有块并行地写到上面的五块磁盘上（图中用灰色标出）。因此，全条带写（full-stripe write）是 RAID-4 写磁盘最高效的方式。

理解了全条带写（full-stripe write），计算 RAID-4 上顺序写的性能就很容易了：有效带宽同样是 $(N-1)\cdot S$ MB/s。尽管在这个过程中校验盘（parity disk）一直在工作，客户端却没有从中得到性能上的好处。

现在来分析随机读的性能。从上图同样可以看出，一组单块随机读会分散到系统的各块数据盘上，但不会落到校验盘（parity disk）上。因此，有效性能是：$(N-1)\cdot R$ MB/s。

我们把随机写留到了最后，它是 RAID-4 最有意思的情况。假设我们想覆盖上例中的块 1。我们可以直接覆盖它，但这会留下一个问题：校验块（parity block）P0 将无法准确反映这个条带（stripe）正确的校验值；在这个例子中，P0 也必须更新。怎样才能既正确又高效地更新它呢？

办法有两种。第一种称为 **additive parity**，做法如下。要算出新校验块（parity block）的值，就并行读入条带（stripe）中其他所有数据块（在这个例子中是块 0、2、3），把它们

 © 2008–23, ARPACI-DUSSEAU

和新块（1）做 XOR，结果就是新的校验块。要完成写操作，接着把新数据和新校验分别写到各自的磁盘上，这两次写同样可以并行进行。

这种技术的问题在于，它的开销随磁盘数量增长，因此在较大的 RAID 中，计算校验需要大量的读操作。于是就有了 **subtractive parity** 方法。

例如，设想下面这串位（4 个数据位，一个校验位）：

C0	C1	C2	C3	P
0	0	1	1	XOR(0,0,1,1) = 0

假设我们想用一个新值覆盖位 C2，新值记作 $C2_{\text{new}}$。subtractive parity 方法分三步。首先，读入 C2 处的旧数据（$C2_{\text{old}} = 1$）和旧校验（$P_{\text{old}} = 0$）。然后，比较旧数据和新数据；如果两者相同（例如 $C2_{\text{new}} = C2_{\text{old}}$），我们就知道校验位也保持不变（即 $P_{\text{new}} = P_{\text{old}}$）。但如果两者不同，就必须把旧校验位翻转成相反的状态，也就是说，如果（$P_{\text{old}} == 1$），P_{new} 就设为 0；如果（$P_{\text{old}} == 0$），P_{new} 就设为 1。这一大堆逻辑可以用 XOR 简洁地表示出来（其中 $\oplus$ 是 XOR 运算符）：

$$P_{\text{new}} = (C_{\text{old}} \oplus C_{\text{new}}) \oplus P_{\text{old}} \tag{38.1}$$

由于我们处理的是块而不是位，所以要对块中的所有位做这个计算（例如每个块 4096 字节，乘以每字节 8 位）。因此，大多数情况下，新块会和旧块不同，新的校验块（parity block）也会和旧的不同。

现在你应该能想明白，什么时候该用 additive parity 计算，什么时候该用 subtractive parity 方法了。想一想，系统中要有多少块磁盘，additive parity 方法执行的 I/O 才会比 subtractive parity 方法少；分界点在哪里？

在这个性能分析中，我们假设使用 subtractive parity 方法。因此，每次写，RAID 都要执行 4 次物理 I/O（两次读，两次写）。现在设想有大量写请求提交给 RAID；RAID-4 能并行执行其中多少个？为了弄清这一点，我们再来看一下 RAID-4 的布局（图 38.6）。

现在设想差不多同时有 2 个小写请求提交给 RAID-4，分别写块 4 和块 13（图中用 * 标出）。这两块数据分别在磁盘 0 和磁盘 1 上，因此对数据的读写可以并行进行，这很好。问题出在校验盘（parity disk）上：两个请求都要读块 4 和块 13 对应的校验块（parity block），即校验块 1 和 3（用 + 标出）。现在应该很清楚了：在这种工作负载下，校验盘（parity disk）成了瓶颈；因此我们有时把它称为基于奇偶校验的 RAID 的**小写入问题**（small-write problem）。所以，即使数据盘可以并行访问，校验盘（parity disk）也让并行无从实现：有了校验盘这个瓶颈，系统的所有写操作都只能串行执行。由于每次逻辑 I/O 都要让校验盘（parity disk）执行两次 I/O（一次读、一次写），只要算出校验盘完成这两次 I/O 的性能，就能得到 RAID-4

磁盘 0	磁盘 1	磁盘 2	磁盘 3	磁盘 4
0	1	2	3	P0
*4	5	6	7	+P1
8	9	10	11	P2
12	*13	14	15	+P3

图 38.6：**示例：写块 4、13 以及相应的校验块**

中小规模随机写的性能：$(R/2)$ MB/s。RAID-4 在小规模随机写下的吞吐量非常糟糕；往系统里加磁盘也不会有所改善。

最后我们来分析 RAID-4 的 I/O 延迟。你现在已经知道，单次读（假设没有故障）只是映射到一块磁盘上，因此其延迟等于单个磁盘请求的延迟。单次写需要两次读，再加两次写；两次读可以并行进行，两次写也可以，因此总延迟大约是单块磁盘的两倍（会有些出入：因为要等两次读都完成，所以会碰上最坏情况的定位时间；但之后的更新不会产生寻道开销，因此定位开销可能比平均水平好）。

38.7 RAID 5 级：轮转奇偶校验

为了（至少部分地）解决小写入问题（small-write problem），Patterson、Gibson 和 Katz 提出了 RAID-5。RAID-5 的工作方式和 RAID-4 几乎完全一样，只是它把校验块（parity block）**轮转**（rotates）到各块磁盘上（图 38.7）。

可以看到，每个条带（stripe）的校验块（parity block）现在轮流分布在各块磁盘上，从而消除了 RAID-4 中校验盘（parity disk）的瓶颈。

RAID-5 分析

RAID-5 的大部分分析和 RAID-4 相同。例如，两个级别的有效容量和容错能力完全相同，顺序读写的性能也一样。单个请求（无论读还是写）的延迟也和 RAID-4 相同。

随机读的性能稍好一些，因为现在可以利用所有磁盘了。最后，随机写的性能比 RAID-4 有明显提高，因为它允许请求之间并行。设想写块 1 和写块 10；这会变成两组请求：针对块 1 及其校验块（parity block）的请求落在磁盘 1 和磁盘 4 上，针对块 10 及其校验块的请求落在磁盘 0 和磁盘 2 上。因此，它们可以并行进行。实际上，我们一般可以假设，有大量随机请求时，所有磁盘都能保持大致均衡的忙碌。如果是这样，小规模写的总带宽就是 $\frac{N}{4} \cdot R$ MB/s。之所以损失了四倍，是因为每次 RAID-5 写操作仍然要产生总共 4 次 I/O 操作，这就是使用基于奇偶校验的 RAID 的代价。

由于 RAID-5 和 RAID-4 基本相同，只是在少数情况下更好，它在市场上几乎完全取代了 RAID-4。唯一没被取代的地方，是那些明知自己只会做大规模写、从而完全避开小写入问题（small-write problem）的系统 [HLM94]；在这种情况下，有时会用 RAID-4，因为它构建起来稍微简单一些。

磁盘 0	磁盘 1	磁盘 2	磁盘 3	磁盘 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

图 38.7：**轮转奇偶校验的 RAID-5**

 © 2008–23, ARPACI-DUSSEAU

	RAID-0	RAID-1	RAID-4	RAID-5
容量	$N \cdot B$	$(N \cdot B)/2$	$(N-1) \cdot B$	$(N-1) \cdot B$
可靠性	0	1（肯定能） $\frac{N}{2}$（运气好时）	1	1
吞吐量				
顺序读	$N \cdot S$	$(N/2) \cdot S$[1]	$(N-1) \cdot S$	$(N-1) \cdot S$
顺序写	$N \cdot S$	$(N/2) \cdot S$[1]	$(N-1) \cdot S$	$(N-1) \cdot S$
随机读	$N \cdot R$	$N \cdot R$	$(N-1) \cdot R$	$N \cdot R$
随机写	$N \cdot R$	$(N/2) \cdot R$	$\frac{1}{2} \cdot R$	$\frac{N}{4}R$
延迟				
读	T	T	T	T
写	T	T	$2T$	$2T$

图 38.8：**RAID 的容量、可靠性和性能**

38.8 RAID 比较：小结

现在，我们在图 38.8 中总结简化后的 RAID 级别比较。注意，为了简化分析，我们省略了许多细节。例如，在镜像系统（mirrored system）中写数据时，平均寻道时间（seek time）比只写一块磁盘时稍长，因为寻道时间是两次寻道中的较大者。因此，对两块磁盘的随机写性能，一般会比对单块磁盘的随机写性能稍差一点。另外，在 RAID-4/5 中更新校验盘（parity disk）时，第一次读旧校验很可能会引起一次完整的寻道和旋转，而第二次写校验则只会引起旋转。最后，与其他方法相比，镜像 RAID（mirrored RAID）的顺序 I/O 要付出 2 倍的性能代价[1]。

不过，图 38.8 确实抓住了本质的差别，对理解各 RAID 级别之间的权衡很有用。在延迟分析中，我们简单地用 T 表示单块磁盘处理一个请求所需的时间。

总之，如果你只要性能、不在乎可靠性，条带化（striping）显然最好。但如果你既要随机 I/O 性能，又要可靠性，镜像最好；付出的代价是容量的损失。如果你的主要目标是容量和可靠性，那么 RAID-5 胜出；付出的代价是小规模写的性能。最后，如果你总是做顺序 I/O，又想把容量最大化，RAID-5 也最合适。

38.9 RAID 的其他有趣问题

思考 RAID 时，还有许多其他有趣的想法可以（也许应该）讨论。下面是我们以后可能会写到的一些内容。

例如，还有许多其他 RAID 设计，包括最初分类中的 2 级和 3 级，以及能承受多块磁盘故障的 6 级 [C+04]。还有磁盘故障时 RAID 怎么办；有时它会准备一块**热备盘**（hot spare）待命，用来顶替坏掉的磁盘。故障期间性能如何？重建坏盘期间的性能又如何？还有更现实的故障模型，要考虑 **latent sector error** 或**块损坏**（block corruption）[B+08]，以及处理这类故障的许多技术，详见数据完整性（data integrity）一章。最后，你甚至可以把 RAID

[1]这里一半的性能代价，假设的是一种简单粗暴的镜像读写模式；更巧妙的方法是向每个镜像（mirror）的不同部分发出大的 I/O 请求，这样就有可能获得全部带宽。想一想，看你能否弄明白其中的道理。

 © 2008–23, ARPACI-DUSSEAU

做成一个软件层：这种**软件 RAID**（software RAID）系统更便宜，但也有别的问题，包括 consistent-update problem [DAA05]。

38.10 小结

我们讨论了 RAID。RAID 把多块独立的磁盘变成一个更大、容量更多、更可靠的单一整体；重要的是，它是透明地做到这一点的，因此上层的硬件和软件基本察觉不到这种变化。

可供选择的 RAID 级别有很多，具体用哪个级别，在很大程度上取决于最终用户看重什么。例如，镜像 RAID（mirrored RAID）简单、可靠，一般能提供不错的性能，但容量代价很高。相比之下，RAID-5 可靠，从容量的角度看也更好，但工作负载中有小规模写时，性能相当差。为特定的工作负载挑选 RAID，并正确设置条带深度（chunk size）、磁盘数量等参数，很有挑战性，至今仍更像一门艺术，而不是一门科学。

参考文献

[B+08] “An Analysis of Data Corruption in the Storage Stack” by Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST ’08, San Jose, CA, February 2008. *Our own work analyzing how often disks actually corrupt your data. Not often, but sometimes! And thus something a reliable storage system must consider.*

[BG88] “Disk Shadowing” by D. Bitton and J. Gray. VLDB 1988. *One of the first papers to discuss mirroring, therein called “shadowing”.*

[CL95] “Striping in a RAID level 5 disk array” by Peter M. Chen and Edward K. Lee. SIGMETRICS 1995. *A nice analysis of some of the important parameters in a RAID-5 disk array.*

[C+04] “Row-Diagonal Parity for Double Disk Failure Correction” by P. Corbett, B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, S. Sankar. FAST ’04, February 2004. *Though not the first paper on a RAID system with two disks for parity, it is a recent and highly-understandable version of said idea. Read it to learn more.*

[DAA05] “Journal-guided Resynchronization for Software RAID” by Timothy E. Denehy, A. Arpaci-Dusseau, R. Arpaci-Dusseau. FAST 2005. *Our own work on the consistent-update problem. Here we solve it for Software RAID by integrating the journaling machinery of the file system above with the software RAID beneath it.*

[HLM94] “File System Design for an NFS File Server Appliance” by Dave Hitz, James Lau, Michael Malcolm. USENIX Winter 1994, San Francisco, California, 1994. *The sparse paper introducing a landmark product in storage, the write-anywhere file layout or WAFL file system that underlies the NetApp file server.*

[K86] “Synchronized Disk Interleaving” by M.Y. Kim. IEEE Transactions on Computers, Volume C-35: 11, November 1986. *Some of the earliest work on RAID is found here.*

[K88] “Small Disk Arrays – The Emerging Approach to High Performance” by F. Kurzweil. Presentation at Spring COMPCON ’88, March 1, 1988, San Francisco, California. *Another early RAID reference.*

[P+88] “Redundant Arrays of Inexpensive Disks” by D. Patterson, G. Gibson, R. Katz. SIGMOD 1988. *This is considered **the** RAID paper, written by famous authors Patterson, Gibson, and Katz. The paper has since won many test-of-time awards and ushered in the RAID era, including the name RAID itself!*

[PB86] “Providing Fault Tolerance in Parallel Secondary Storage Systems” by A. Park, K. Balasubramaniam. Department of Computer Science, Princeton, CS-TR-O57-86, November 1986. *Another early work on RAID.*

[SG86] “Disk Striping” by K. Salem, H. Garcia-Molina. IEEE International Conference on Data Engineering, 1986. *And yes, another early RAID work. There are a lot of these, which kind of came out of the woodwork when the RAID paper was published in SIGMOD.*

[S84] “Byzantine Generals in Action: Implementing Fail-Stop Processors” by F.B. Schneider. ACM Transactions on Computer Systems, 2(2):145–154, May 1984. *Finally, a paper that is not about RAID! This paper is actually about how systems fail, and how to make something behave in a fail-stop manner.*

作业（模拟）

本节介绍 `raid.py`，这是一个简单的 RAID 模拟器，你可以用它来巩固对 RAID 系统工作原理的理解。详情请见 README。

问题

1. 用模拟器做一些基本的 RAID 映射测试。用不同的级别（0、1、4、5）运行，看看你能否算出一组请求的映射。对于 RAID-5，看看你能否弄清楚左对称（left-symmetric）布局和左不对称（left-asymmetric）布局的区别。用一些不同的随机种子，生成和上面不同的问题。
2. 和第一题一样，但这次用 `-C` 改变条带深度（chunk size）。条带深度（chunk size）如何改变映射?
3. 和上面一样，但用 `-r` 选项把每个问题反过来。
4. 现在使用反向选项，但用 `-S` 选项增大每个请求的大小。试着指定 8k、12k 和 16k 的大小，同时改变 RAID 级别。请求变大时，底层的 I/O 模式有什么变化？一定也要用顺序负载（`-W sequential`）试试；请求多大时，RAID-4 和 RAID-5 的 I/O 效率会高得多?
5. 使用模拟器的计时模式（`-t`），用 4 块磁盘，在改变 RAID 级别的同时，估算对 RAID 做 100 次随机读的性能。
6. 和上面一样，但增加磁盘的数量。磁盘数量增加时，各 RAID 级别的性能如何扩展?
7. 和上面一样，但全部改成写（`-w 100`），不用读。现在各 RAID 级别的性能如何扩展？你能粗略估算出完成这 100 次随机写的工作负载需要多长时间吗?
8. 最后再运行一次计时模式，但这次用顺序负载（`-W sequential`）。性能如何随 RAID 级别变化，读和写时又有什么不同？改变每个请求的大小时呢？使用 RAID-4 或 RAID-5 时，每次应该向 RAID 写多大的数据?

 © 2008–23, ARPACI-DUSSEAU